

Deep Technical Review — Segment Coronary Artery Pipeline

Claude Code (claude-sonnet-4-6)

March 2026

Contents

1	Deep Technical Review — Segment Coronary Artery Pipeline	1
1.1	1. Project Overview	1
1.2	2. File-by-File Findings	2
1.2.1	2.1 medis.py — Cohort Overview Builder	2
1.2.2	2.2 medismask.py — Plaque Mask Generation	3
1.2.3	2.3 medis_to_json.py and medis_to_gii.py — Contour → Mesh Converters	7
1.2.4	2.4 medis_to_stl.py — Contour → STL Exporter	8
1.2.5	2.5 medis_to_cpr.py — Curved Planar Reformation	10
1.2.6	2.6 transform_to_cpr.py — World-to-CPR Coordinate Transformer	12
1.2.7	2.7 vessel-viewer.html — Browser Visualisation	14
1.2.8	2.8 md2pdf.sh and install_md2pdf.sh — PDF Build Toolchain	16
1.3	3. Cross-Cutting Architecture Issues	17
1.3.1	3.1 Code Duplication (Critical)	17
1.3.2	3.2 Hard-Coded Paths	17
1.3.3	3.3 Missing Infrastructure	17
1.3.4	3.4 Coordinate System Documentation	18
1.4	4. Execution List	18
1.4.1	Phase 1 — Correctness (Blocking Issues)	18
1.4.2	Phase 2 — Performance	18
1.4.3	Phase 3 — Maintainability	19
1.5	5. Summary Scorecard	19

1 Deep Technical Review — Segment Coronary Artery Pipeline

1.1 1. Project Overview

This repository implements a research pipeline for coronary artery morphometry. It ingests MEDIS QCT software export files (plain-text contour files) and converts them into four downstream formats:

Output	File	Purpose
Binary NIfTI plaque masks	medismask.py	Quantitative imaging / AI training
GlTF 3D meshes	medis_to_gii.py	CFD / NiiVue overlay

Output	File	Purpose
NiiVue JSON point clouds	<code>medis_to_json.py</code>	Browser visualisation
STL meshes	<code>medis_to_stl.py</code>	3D printing / CFD
CPR volumes + frame	<code>medis_to_cpr.py</code>	Straightened vessel viewing
CPR-space meshes	<code>transform_to_cpr.py</code>	Overlay on CPR
Overview CSVs	<code>medis.py</code>	Cohort-level bookkeeping

The tooling layer consists of a NiiVue-based HTML viewer (`vessel-viewer.html`) and a Pandoc/XeLaTeX PDF builder (`md2pdf.sh`).

1.2 2. File-by-File Findings

1.2.1 2.1 `medis.py` — Cohort Overview Builder

Purpose: Scans MEDIS export directories for two readers (Eser, Robin), extracts patient ID and vessel name from file headers, joins against a UID CSV, and writes summary CSVs and patient-ID text files.

Findings:

Severity	Issue
HIGH	Hard-coded Windows path <code>C:\Users\lukass\Desktop\personal\data\fl</code> at module level. Script cannot be run on any other machine without editing source.
MEDIUM	<code>load_uids()</code> deduplication sorts by Source descending and calls <code>drop_duplicates(keep='first')</code> to prefer “Session” over “Button”. This relies on lexicographic sort order of the string values — fragile and undocumented.
MEDIUM	Three entry points (<code>main</code> , <code>main2</code> , <code>main3</code>) with <code>main3</code> active via <code>if __name__ == "__main__"</code> and the others commented out. This creates confusion about the intended execution order.

Severity	Issue
LOW	No requirements.txt, pyproject.toml, or environment.yml. Users must infer dependencies from imports (pandas, pathlib).

What I would change and why:

```
# Instead of module-level hard-coded paths, use CLI args:
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--flow-root', required=True)
    ...

# Instead of three main() functions, use subcommands:
# python medis.py scan      -> runs scan + per-reader CSVs
# python medis.py merge     -> merges into unified CSV
# python medis.py summarise -> statistics
```

Add a `pipeline()` function that calls all three steps in order, document the dependency chain clearly.

1.2.2 2.2 medismask.py — Plaque Mask Generation

Purpose: The core image-processing module. Loads a DICOM CT series, loads MEDIS contour TXT files, and rasterises the vessel wall annulus (wall minus lumen) into a 3D binary NIfTI mask. Two methods: polygon filling and radial interpolation.

Findings:

Severity	Issue	Location
HIGH	Hard-coded Windows path r"C:\Users\lukass\Downloads\plaque" inside run_plaque_pipeline(). CLI parsing exists but the pipeline function ignores CLI arguments — it always uses the internal path.	run_plaque_pipeline() L522
HIGH	Massive performance bug in create_plaque_mask_polygon(): pixels are rasterised into a 2D index-space ring, then each pixel is individually converted <i>back</i> to a physical point, then painted into a new mask one voxel at a time via Python loops. For a typical CT (512×512×300), this means millions of iterations in Python with no vectorisation.	L370–L398
HIGH	Bare except: pass on L379 silently swallows all exceptions, including KeyboardInterrupt. This is especially dangerous in coordinate transforms where silent failures mean wrong masks with no warning.	L379

Severity	Issue	Location
MEDIUM	Mask is allocated as <code>sitk.sitkInt16</code> (2 bytes/voxel) then cast to <code>sitk.sitkUInt8</code> . The <code>Int16</code> allocation wastes 2× memory during processing with no benefit.	L384, L192
MEDIUM	<code>match_lumen_wall()</code> produces incorrect indices when <code>diff > ves - sel_wall.shape[0]</code>: the formula <code>q = ves - sel_wall.shape[0]</code> <code>// diff</code> yields <code>q=0</code> , and then <code>i * 0 - 1 = -1</code> for all iterations, deleting the last row repeatedly instead of evenly distributed rows.	L118–L121
MEDIUM	<code>np.squeeze()</code> bug in <code>load_ct_series_to_nifti()</code>: <code>np.squeeze(array)</code> removes <i>all</i> singleton dimensions. If the CT happens to have a 512×1×Z volume (edge case), it will silently collapse the wrong axis. Should use explicit indexing: <code>array[:, :, :, 0]</code> or <code>array[0]</code> .	L492
LOW	Debug progress prints are mixed with application output (no logging levels).	Throughout

Severity	Issue	Location
LOW	n_points parameter in the radial method is a small int with no input validation (could be 0 or negative).	L180

What I would change and why:

Replace the double coordinate-transformation in `create_plaque_mask_polygon()` with direct numpy array indexing:

```
# Current approach (slow): 2D indices -> physical points -> re-index into mask
# Better approach:
mask_array = sitk.GetArrayFromImage(mask) # (z, y, x)
for z_idx, ring_2d in z_index_to_mask.items():
    if 0 <= z_idx < mask_array.shape[0]:
        # ring_2d is already in (y, x) = (row, col) order
        mask_array[z_idx] |= ring_2d # single vectorised operation per z-slice
mask = sitk.GetImageFromArray(mask_array)
mask.CopyInformation(ct_image)
```

This eliminates the physical↔index roundtrip entirely and replaces $O(N_{\text{pixels}})$ Python loops with a single NumPy bitwise-OR per slice.

Fix the bare except:

```
# Instead of bare except:
except (RuntimeError, OverflowError):
    pass
```

Fix `match_lumen_wall()`:

```
def match_lumen_wall(lumen, vessel_wall):
    n_lumen = lumen.shape[0]
    n_wall = vessel_wall.shape[0]
    if n_lumen == n_wall:
        return vessel_wall
    # Use linear interpolation resampling instead of deletion/duplication
    indices = np.linspace(0, n_wall - 1, n_lumen)
    return np.array([vessel_wall[int(round(i))]] for i in indices))
```

1.2.3 2.3 medis_to_json.py and medis_to_gii.py — Contour → Mesh Converters

Purpose: Parse MEDIS contour TXT files, resample each contour ring to a uniform point count via cubic spline interpolation, align rings to prevent mesh twisting, and export as either NiiVue JSON (connectome format) or GiftI mesh with end caps.

Findings:

Severity	Issue
HIGH	parse_medis_contours() , resample_polygon() , and align_contours() are copy-pasted verbatim between both files (and again in medis_to_stl.py). Any bug fix must be applied to three places.
HIGH	The nifti_cta_path parameter is accepted by both convert_medis_to_json() and convert_medis_to_gii() but is never used inside either function. The CTA is not needed for mesh creation. This is misleading and clutters the API.
MEDIUM	Debug print statements left in production code: print(f"Debug: Read {len(lines)} lines from file"), print(f"Debug: Parsing contour {contour_count} at line {i}"). Should use logging.debug().
MEDIUM	contours_to_niivue_json() creates inter-ring edges with a hardcoded stride of 4: for i in range(0, target_points, 4). For target_points=16 this creates very dense connectivity; for target_points=256 it is very sparse. The stride should be proportional to target_points.

Severity	Issue
MEDIUM	<code>align_contours()</code> aligns each ring's start point to the closest point in the previous ring. This greedy approach can accumulate twist for helical vessels. A signed-angle criterion (preserving orientation) would be more robust.
LOW	<code>contours_to_niivue_json()</code> sets <code>colorValue</code> and <code>sizeValue</code> to uniform 1.0 for all points. These fields are useful (e.g. colour-coding by wall thickness) but are not populated.
LOW	<code>GiftiMetaData.from_dict()</code> is deprecated in nibabel ≥4.0. Use <code>GiftiMetaData(**dict)</code> instead.

What I would change and why:

Create a shared `medis_utils.py` module:

```
# medis_utils.py
def parse_medis_contours(filepath: str) -> tuple: ...
def resample_polygon(polygon: np.ndarray, n_points: int) -> np.ndarray: ...
def align_contours(contours: list) -> list: ...
```

All three converter scripts would then `from medis_utils import ...`. This is the most impactful single change across the codebase — it eliminates three copies of ~80 lines each.

1.2.4 2.4 `medis_to_stl.py` — Contour → STL Exporter

Purpose: Generate STL tube meshes (lumen and vessel wall) from MEDIS contour files. Uses the same resample→align→mesh pipeline as the GiftI converter, but writes STL via the `stl` library.

Findings:

Severity	Issue
HIGH	create_tube_mesh() has a vertex index bug : it addresses vertices as $i * \max(n1, n2) + j$, but the <code>all_points</code> list is built by appending each contour consecutively with their <i>actual</i> sizes. When $n1 \neq n2$, the stride $\max(n1, n2)$ creates out-of-range or wrong indices. The function is unused in <code>process_file()</code> (which calls <code>create_tube_mesh_simple()</code> instead), but its presence is a maintenance hazard.
HIGH	Hard-coded Windows path <code>r"C:\Users\lukass\Desktop\personal\github</code> in <code>main()</code> .
MEDIUM	<code>resample_polygon()</code> in this file lacks the <code>if perimeter < 1e-10:</code> <code>return polygon</code> guard present in the other converters. For degenerate contours (all points coincident), <code>CubicSpline</code> will raise a <code>ValueError</code> .
MEDIUM	<code>len(points) > 0</code> threshold in <code>parse_contours()</code> accepts contours with 1–2 points — insufficient to form a polygon. Should be <code>> 2</code> , consistent with <code>medis_to_json.py</code> .
LOW	<code>create_tube_mesh()</code> is dead code. Remove it to avoid confusion with the working <code>create_tube_mesh_simple()</code> .
LOW	Same duplication as above: <code>parse_contours()</code> , <code>resample_polygon()</code> , <code>align_contours()</code> re-implemented here.

What I would change and why:

Remove `create_tube_mesh()` entirely (it is never called and is buggy). Add the missing perimeter guard to `resample_polygon()`. Port to use `medis_utils.py` as described above.

1.2.5 2.5 `medis_to_cpr.py` — Curved Planar Reformation

Purpose: Compute cross-sectional and longitudinal CPR volumes from MEDIS lumen contours. Extracts centerline as contour centroids, fits a Rotation Minimizing (Bishop) Frame, then samples the CTA volume on cross-sectional planes perpendicular to the centerline.

Findings:

Severity	Issue
HIGH	<code>slice_distances</code> is parsed but never used. The CPR is constructed purely from the spatial ordering of contours in the file, not from the actual <code>SliceDistance</code> values. If MEDIS exports contours out of spatial order, the centerline will be scrambled. Should sort by <code>SliceDistance</code> before constructing the centerline.
MEDIUM	No validation that lumen contours are spatially ordered. A simple check: if consecutive centroid distances are non-monotone, warn and optionally re-sort.
MEDIUM	<code>save_nifti()</code> sets the NIfTI origin to $(0, 0, 0)$. This is intentional (CPR space is self-contained), but it means that if two CPR volumes are loaded together in a viewer, their origins will conflict. Adding a unique identifier to the description metadata would help.

Severity	Issue
MEDIUM	create_cross_sectional_cpr() and create_longitudinal_cpr() sample the same data but are implemented as separate functions with duplicated sampling loops. The only difference is array layout and spacing tuple order. A single function with an orientation parameter would halve the code.
LOW	target_spacing = min(cta_spacing) uses the finest voxel spacing as target CPR resolution. For anisotropic CT (e.g. $0.3 \times 0.3 \times 0.625$ mm), this oversamples the CPR along the vessel axis and produces unnecessarily large files. A parameter for independent cross-section and along-vessel resolution would be better.
LOW	No progress indication during the slice-by-slice CPR sampling (can take 1–2 minutes for 500 centerline points).

What I would change and why:

Use the SliceDistance values to sort contours:

```
# After parsing contours and slice_distances:
lumen_with_dist = sorted(
    zip(slice_distances['Lumen'], contours['Lumen']),
    key=lambda x: x[0]
)
sorted_distances, sorted_contours = zip(*lumen_with_dist)
centerline = extract_centerline(list(sorted_contours))
```

Merge the two CPR sampling functions:

```
def create_cpr(volume, spacing, origin, centerline, normals, binormals,
               cross_section_size_mm=30.0, pixel_resolution=None,
               orientation='cross'): # or 'longitudinal'
    ...
```

```

if orientation == 'cross':
    cpr_volume[..., i] = sampled
else:
    cpr_volume[i, ...] = sampled

```

1.2.6 2.6 transform_to_cpr.py — World-to-CPR Coordinate Transformer

Purpose: Given a pre-computed CPR frame (centerline + Bishop frame vectors), transform GlTFI mesh vertices or JSON point cloud coordinates from world space to CPR voxel space for overlay visualization.

Findings:

Severity	Issue
HIGH	$O(N \times M)$ complexity in <code>world_to_cpr()</code>: for every point in the mesh, computes distance to every centerline point. For a fine mesh with 10,000 vertices and a 500-point centerline, this is 5 million Euclidean distance calculations in a Python loop. Should use <code>scipy.spatial.cKDTree</code> .
MEDIUM	Sub-index precision is computed but discarded: <code>best_s</code> is computed as a float representing sub-integer position along the centerline, but then rounded to an integer before being stored in <code>cpr_points[i, 2]</code> . The <code>s</code> coordinate is thus quantized to integer centerline indices, making the along-vessel mapping coarse. The float value should be retained.

Severity	Issue
MEDIUM	cpr_spacing=0.25 and cross_section_size=120 are hardcoded defaults in transform_gifti_to_cpr() and transform_json_to_cpr(). These must match exactly the values used during CPR construction in medis_to_cpr.py. If they drift, the mesh overlay will be spatially misaligned. Should read these from the frame JSON.
LOW	batch_transform() extracts vessel name from frame filename using string splitting. The parts[-1] heuristic breaks if the vessel name itself contains an underscore (e.g. a hypothetical "OM_1").

What I would change and why:

Replace the nearest-centeline search with a KD-tree:

```
from scipy.spatial import cKDTree

def world_to_cpr(world_points: np.ndarray, frame: dict) -> np.ndarray:
    centerline = frame['centerline']
    tree = cKDTree(centerline)
    _, s_indices = tree.query(world_points)
    # then compute local (u, v) offsets vectorised
    P = centerline[s_indices]          # Nx3
    D = world_points - P               # Nx3
    N = frame['normals'][s_indices]    # Nx3
    B = frame['binormals'][s_indices]  # Nx3
    u = np.einsum('ij,ij->i', D, N)
    v = np.einsum('ij,ij->i', D, B)
    return np.column_stack([u, v, s_indices.astype(np.float32)])
```

This is approximately 200× faster than the current implementation for typical mesh sizes.

Store CPR construction parameters in frame JSON and read them in the transformer:

```
{
  "centerline": [...],
```

```

    "cpr_spacing_mm": 0.3,
    "cross_section_size_pixels": 100,
    ...
}

```

1.2.7 2.7 vessel-viewer.html — Browser Visualisation

Purpose: NiiVue-based single-page viewer for CTA volumes with GlTFI mesh overlays. Supports World Space and CPR Space modes, with controls for vessel selection, mesh opacity, shaders, and view orientation.

Findings:

Severity	Issue
HIGH	CDN dependency without version pinning: @niivue/niivue@latest will silently pull breaking API changes on any reload after a new NiiVue release. Observed API differences between NiiVue 0.x and 1.x include renamed methods. Pin to a specific version: @niivue/niivue@0.44.0.
MEDIUM	Points removal is not implemented: Unchecking “Inner Points” or “Outer Points” calls nv.drawScene() but does not remove the connectome. This is noted as a TODO in a comment but left as a broken feature in production.
MEDIUM	getMeshPath() accepts an unused suffix = '' parameter that appears to have been left from an earlier refactor. Dead parameter creates false expectations.
MEDIUM	No error recovery for failed volume/mesh loads. After an error, the UI state (checkboxes, dropdowns) is inconsistent with what is actually loaded. Should reset checkboxes on load failure.

Severity	Issue
LOW	Hard-coded single patient 01-BER-0088. Adding patients requires editing HTML. A JSON manifest file or URL parameter would allow the viewer to be used for any patient.
LOW	Emoji characters in section headers (👤, 🏠, 📄) may not render in all WebGL environments or when the page is printed.
LOW	vessel-viewer-minimal.html and vessel-viewer-simple.html are variants of the same viewer. They likely share substantial duplicated code.

What I would change and why:

Pin the NiiVue version:

```
<script src="https://cdn.jsdelivr.net/npm/@niivue/niivue@0.44.0/dist/niivue.umd.js">
```

Load patient/vessel configuration from a JSON manifest so the viewer is data-driven:

```
// manifest.json
{
  "patients": [
    { "id": "01-BER-0088", "vessels": ["LAD", "RCA", "LCX", "D1"] },
    { "id": "02-BER-0099", "vessels": ["LAD", "RCA"] }
  ]
}
```

Implement proper points removal:

```
// Track connectome IDs for removal
let loadedConnectomeIds = { inner: null, outer: null };

async function loadPoints(type) {
  if (loadedConnectomeIds[type]) {
    nv.removeConnectome(loadedConnectomeIds[type]);
    loadedConnectomeIds[type] = null;
  }
  // ... load and store ID
}
```

1.2.8 2.8 md2pdf.sh and install_md2pdf.sh — PDF Build Toolchain

Purpose: Pandoc + XeLaTeX pipeline for converting Markdown documents to PDF (and optionally DOCX/TEX). The installer handles multi-platform package management and font detection.

Findings:

Severity	Issue
MEDIUM	eval \$INSTALL_CMD \$PKG_LIST in <code>install_md2pdf.sh</code> uses <code>eval</code> with shell variables. While the values are hardcoded here, this pattern is insecure: if <code>\$PKG_LIST</code> were ever set externally (e.g. via environment variable), it could execute arbitrary code. Use array-based execution instead.
MEDIUM	The font detection for Noto Sans in <code>md2pdf.sh</code> has a redundant branch: lines 112–116 check <code>fc-list grep -qi "Noto Sans"</code> twice with slightly different conditions. The inner <code>if/elif</code> is unreachable as written.
LOW	<code>md2pdf.sh</code> uses <code>set -e</code> but does not use <code>set -u</code> (undefined variable checking) or <code>set -o pipefail</code> . Pipeline failures in commands connected by <code> </code> are silently ignored.
LOW	The bibliography search paths include <code>\$WORKSPACE/flow/references.bib</code> — a path that doesn't exist in this repository. Harmless but indicates leftover from a different project.

What I would change and why:

Replace `eval` with an array:

```
# Instead of:
eval $INSTALL_CMD $PKG_LIST
```



```
# Use:
INSTALL_ARRAY=($INSTALL_CMD) # or build a proper array
"${INSTALL_ARRAY[@]}" $PKG_LIST
```

Add `set -uo pipefail` to both scripts. Fix the Noto Sans font check duplicate branch.

1.3 3. Cross-Cutting Architecture Issues

1.3.1 3.1 Code Duplication (Critical)

The following three functions are copy-pasted across **three separate files**:

Function	medis_to_json.py	medis_to_gii.py	medis_to_stl.py
<code>parse_medisContours()</code>	☐	☐	☐ (as <code>parse_contours()</code>)
<code>resample_polygon()</code>	☐	☐	☐
<code>align_contours()</code>	☐	☐	☐

A bug fix in any of these functions must be manually applied three times. The `parse_contours()` variant in `medis_to_stl.py` already diverges (missing metadata extraction), demonstrating that the copies are drifting. **Create `medis_utils.py` and import from it.**

1.3.2 3.2 Hard-Coded Paths

Four files contain hard-coded Windows paths for a specific developer machine:

File	Hard-Coded Path
<code>medis.py</code>	<code>C:\Users\lukass\Desktop\personal\data\flow</code>
<code>medismask.py</code>	<code>C:\Users\lukass\Downloads\plaque</code>
<code>medis_to_stl.py</code>	<code>C:\Users\lukass\Desktop\personal\github\flow\data</code>
<code>vessel-viewer-README.md</code>	<code>c:\Users\lukass\Documents\GitHub\segment</code>

None of these scripts can be run by a collaborator or on a server without modifying source code. Use CLI arguments, environment variables, or a config file.

1.3.3 3.3 Missing Infrastructure

Item	Status
<code>requirements.txt</code> / <code>pyproject.toml</code>	Missing

Item	Status
Unit tests	None
CI configuration	None
.gitignore	Present (not reviewed)
Logging configuration	None (raw print throughout)

1.3.4 3.4 Coordinate System Documentation

The pipeline involves four coordinate systems:

1. **Physical (world) space** — millimetres, origin from DICOM header
2. **Voxel (index) space** — integer indices, origin (0,0,0)
3. **CPR space** — (U, V, S) in mm, origin at centerline start
4. **CPR voxel space** — (u_px, v_px, s_idx), centred in cross-section

Functions do not document which coordinate system their inputs and outputs use. Adding parameter annotations (e.g. `world_points: np.ndarray # Nx3, physical mm`) or coordinate system tags would prevent subtle transform bugs.

1.4 4. Execution List

The following tasks are prioritised by impact. Items marked **[BLOCKING]** prevent correctness; others improve performance or maintainability.

1.4.1 Phase 1 — Correctness (Blocking Issues)

1. **[BLOCKING] Sort contours by SliceDistance in `medis_to_cpr.py`**
 - Without this, CPR construction is undefined for non-sorted MEDIS exports.
2. **[BLOCKING] Fix bare `except: pass` in `medismask.py`**
 - Replace with `except (RuntimeError, OverflowError): pass`. Prevents silent corruption.
3. **[BLOCKING] Fix `match_lumen_wall()` index calculation**
 - Use linear resampling instead of deletion with a computed stride.
4. **[BLOCKING] Fix vertex indexing in `create_tube_mesh()` in `medis_to_stl.py`**
 - Either fix the function or remove it (it is dead code).
5. **[BLOCKING] Add perimeter guard to `resample_polygon()` in `medis_to_stl.py`**
 - Prevents `ValueError` from `CubicSpline` on degenerate contours.
6. **[BLOCKING] Pin NiiVue CDN version in `vessel-viewer.html`**
 - Prevents silent breakage on future NiiVue releases.

1.4.2 Phase 2 — Performance

7. **Replace double coordinate transform in `create_plaque_mask_polygon()`**

- Use direct numpy array indexing. Expected 100–1000× speedup.
8. **Replace Python loop with cKDTree in `world_to_cpr()`**
 - Expected 200× speedup for typical mesh sizes.
 9. **Retain float `best_s` in CPR coordinate mapping**
 - Improves along-vessel coordinate precision.
 10. **Merge duplicate CPR sampling loops in `medis_to_cpr.py`**
 - Reduces code by ~50 lines and ensures both views always stay in sync.

1.4.3 Phase 3 — Maintainability

11. **Create `medis_utils.py` with shared contour utilities**
 - Extract `parse_medis_contours()`, `resample_polygon()`, `align_contours()`.
 - Update all three converter scripts to import from it.
 12. **Replace all hard-coded paths with CLI arguments**
 - `medis.py`, `medismask.py`, `medis_to_stl.py` all need this.
 13. **Add `requirements.txt` / `pyproject.toml`**
 - Minimum: `numpy`, `scipy`, `nibabel`, `SimpleITK`, `numpy-stl`, `pandas`.
 14. **Replace `print()` with logging**
 - Set `logging.basicConfig(level=logging.INFO)` in `main()`.
 - Use `logging.debug()` for debug-only prints.
 15. **Implement points removal in `vessel-viewer.html`**
 - Track and remove connectome objects when checkboxes are unchecked.
 16. **Replace `GiftiMetaData.from_dict()` with `GiftiMetaData(**dict)`**
 - `from_dict` is deprecated in `nibabel` ≥4.0.
 17. **Remove dead `nifti_cta_path` parameter from `convert_medis_to_json()` and `convert_medis_to_gii()`**
 18. **Replace `eval` in `install_md2pdf.sh` with array execution**
 19. **Add `set -uo pipefail` to both shell scripts**
 20. **Write unit tests for `resample_polygon()`, `align_contours()`, `world_to_cpr()`**
 - These are pure functions with well-defined mathematical properties that are easy to test.
-

1.5 5. Summary Scorecard

Category	Score	Notes
Correctness	5/10	Blocking bugs in masking, CPR sorting, STL indexing
Performance	4/10	Two critical $O(N^2)$ / loop-based bottlenecks
Maintainability	4/10	Significant code duplication, no tests
Documentation	7/10	Good inline docs and <code>medismask.md</code> ; coord systems undocumented
Portability	2/10	Four hard-coded Windows paths
Security	7/10	Only minor shell scripting concern

The pipeline demonstrates sophisticated medical imaging knowledge and well-chosen algorithms (Bishop Frame, cubic spline resampling, polygon fill with hole-filling). The main weaknesses are engineering rather than algorithmic: duplicated utilities, hard-coded paths, and two performance-critical loops that should be vectorised.